DATA MINING

(SECP2753)

PROJECT 2

Lecturer:

ASSOC DR ROLIANA BINTI IBRAHIM

Group Members:

No	Name	Matric No
1	MIKHAIL BIN YASSIN	A21EC0053
2	PRAVINRAJ A/L SIVABATHI	A23CS0171
3	DHESHIEGHAN A/L SARAVANA MOORTHY	A23CS0072

Table of Content

INTRODUCTION

Obesity is a rising global health concern that significantly increases the risk of chronic diseases such as diabetes, cardiovascular conditions, and certain cancers. According to the World Health Organization (WHO), the global obesity rate has nearly tripled since 1975. In Malaysia, the issue is particularly pressing, with national health surveys indicating that nearly half the adult population is either overweight or obese. These trends highlight the need for improved data-driven strategies to better understand the contributing factors and identify at-risk groups.

In this project, we explore a dataset related to individual health and lifestyle habits to uncover patterns in obesity risk. Unlike classification tasks, which rely on labeled outcomes, this analysis uses clustering techniques to discover natural groupings within the data without predefined labels. By clustering individuals based on lifestyle, diet, and demographic attributes, we aim to gain deeper insight into how certain behaviors and profiles correlate with obesity tendencies.

This approach is useful in real-world scenarios where labels such as "obese" or "non-obese" may not be available or reliable, but data-driven segmentation is still needed to guide interventions, wellness programs, and public health campaigns.

OVERVIEW

Dataset Link : https://www.kaggle.com/datasets/ruchikakumbhar/obesity-prediction

The dataset used for this analysis focuses on identifying health and lifestyle patterns that may be associated with obesity. It was sourced from Kaggle, a leading data science platform, and is widely used in research and academic studies related to health analytics. Originally compiled to support obesity classification research, the dataset is also highly suitable for unsupervised learning techniques such as clustering due to its rich combination of numerical and categorical variables.

The dataset contains 17 attributes and approximately 500 records, each representing an individual. All entries are clean, complete, and free of missing or duplicate values. The attributes cover a range of demographic, dietary, and behavioral features that may influence a person's risk of obesity. These variables provide the necessary diversity to form meaningful clusters using machine learning techniques.

Below is the list of attributes included in the dataset:

No	Attribute	Description
1	Gender	Gender of the individual
2	Age	Age of the individual in years
3	Height	Height in meters
4	Weight	Weight in kilograms
5	Family_history	Whether a family member has been overweight
6	FAVC	High-calorie food consumption frequency
7	FCVC	Vegetable consumption frequency
8	NCP	Number of main meals per day
9	CAEC	Snacking habits between meals
10	SMOKE	Smoking status
11	CH2O	Daily water intake
12	SCC	Calorie monitoring habits
13	FAF	Frequency of physical activity
14	TUE	Time spent using technology

15	CALC	Alcohol consumption frequency
16	MTRANS	Preferred mode of transportation
17	Obesity_level	Obesity level (used for comparison, not for training in clustering)

1.2 Objectives of the Analysis

The primary goal of this project is to use **unsupervised machine learning (clustering)** to discover hidden patterns in individuals' health and lifestyle data. Unlike supervised classification, this analysis does not use obesity labels to train the model. Instead, it identifies groups of individuals with similar behavior profiles and investigates whether these groupings reveal meaningful health-related patterns.

Specifically, the objectives are:

- **To segment individuals into meaningful clusters**
  Group individuals based on similarities in lifestyle, diet, physical activity, and demographic attributes using clustering algorithms such as K-Means or Hierarchical Clustering.

- **To analyze behavioral and demographic patterns within clusters**
  Examine whether clusters show clear distinctions in habits such as fast food consumption, physical activity, water intake, and alcohol use. Explore how factors like gender, age, and transportation choice contribute to these clusters.

- **To compare clusters with known obesity levels**
  Even though clustering is unsupervised, the labeled "Obesity_level" column will be used for post-clustering interpretation. This helps validate whether the clusters align meaningfully with real-world health outcomes.

By achieving these objectives, this analysis aims to help healthcare stakeholders better understand patterns of behavior that may lead to obesity, even in situations where medical labels are not available or clearly defined.

1.3 Business Insights to be Explored

Understanding patterns of behavior associated with obesity can provide valuable insights for health practitioners, policymakers, and wellness platforms aiming to reduce obesity rates. Through clustering analysis, this project seeks to uncover hidden lifestyle profiles and demographic segments that may not be obvious through traditional classification methods.

The key business and health-related insights we aim to extract include:

- Uncovering lifestyle-driven groups
  Clustering individuals based on shared habits (e.g., frequent snacking, lack of physical activity, low water intake) can reveal behavioral profiles that correlate with weight gain or healthy living. These profiles can be used to develop targeted awareness programs and lifestyle interventions.

- Identifying high-risk clusters
  Post-clustering, we will compare the clusters with known obesity levels (from the "Obesity_level" attribute) to determine which clusters contain more individuals with overweight or obese status. This helps in identifying which lifestyle combinations are most commonly associated with obesity.

- Demographic segmentation for outreach
  Clusters will be examined for demographic composition (e.g., age, gender, transport method). This allows health organizations to tailor educational campaigns for specific audiences, like young sedentary adults or tech-heavy users with poor dietary habits.

By using clustering rather than labels, the analysis avoids assumptions and instead finds data-driven segments, enabling more adaptive and personalized health strategies in real-world applications.

1.4 Clustering Models Selected for the Analysis

For this project, we selected clustering algorithms that are effective in grouping data without predefined labels. These algorithms are widely used in unsupervised learning tasks and are particularly suitable for discovering hidden structure in health-related datasets:

1.4.1 K-Prototypes Clustering

K-Prototypes is an extension of K-Means that is designed to handle datasets containing both numerical and categorical features. Unlike K-Means, which only works with numerical data by calculating Euclidean distance, K-Prototypes combine Euclidean distance for numerical variables and a dissimilarity measure (like matching dissimilarity) for categorical variables. It assigns each data point to the nearest cluster prototype based on a combined cost function.

- **Advantages:**
  K-Prototypes is suitable for mixed-type data, making it more flexible than K-Means in real-world scenarios. It is relatively fast, can handle large datasets, and allows users to specify the number of clusters (k) as in K-Means.

- **Limitations:**
  It still requires predefining the number of clusters (k), and results can be sensitive to the initial placement of cluster prototypes. Additionally, choosing the right balance (gamma) between numerical and categorical influence can be challenging.

Why not K-Means?
K-Means does not support categorical data, as it relies on averaging feature values, which is not meaningful for categories (e.g., 'Male' and 'Female' can't be averaged). Using K-Means on mixed or purely categorical data would lead to invalid or misleading clusters. This is why K-Prototypes is a better choice for datasets that include both types of variables.

1.4.3 HDBSCAN (Hierarchical Density-Based Spatial Clustering)

HDBSCAN is an advanced density-based clustering algorithm that builds on DBSCAN by allowing clusters of varying densities and creating a hierarchy of potential clusters. It works particularly well when the data is mostly **numerical** or already transformed into a suitable distance space.

In the case of datasets with **mixed data types** like **numerical, categorical, and ordinal features**, HDBSCAN can still be used **if a proper distance matrix is calculated**. This is often done by combining different distance measures (e.g., Gower distance or using encoded versions of categorical/ordinal features). HDBSCAN then uses this precomputed distance matrix to group similar points and identify noise.

Advantages (compared to DBSCAN):

- **Handles varying densities** better than DBSCAN, which is important for real-world data where cluster compactness isn't uniform.
- **No need to choose a fixed epsilon (ε)**, which is hard to determine in mixed-type data.
- **More stable clusters** and better handling of noise, especially when using a distance metric that accounts for different data types.
- **Flexible with precomputed distances**, making it a better fit for **mixed datasets** when the right distance metric is used.

Limitations:

- **Still requires preprocessing**: For HDBSCAN to work well with categorical and ordinal features, the data usually needs to be encoded or transformed carefully (e.g., ordinal encoding, one-hot encoding, or Gower distance).
- **Computationally intensive** on large datasets when using distance matrices.
- **Does not support raw categorical variables directly**, unlike K-Prototypes, which is built for that.

Why not DBSCAN?

While DBSCAN can detect arbitrarily shaped clusters and doesn’t require the number of clusters beforehand, it **only works well with numerical data**, where distances like Euclidean are meaningful. When applied to **mixed-type datasets**, DBSCAN struggles because:

- It requires a single ε value, which doesn’t generalize well across features with different scales and types.

- It’s highly sensitive to how categorical and ordinal features are encoded which could lead to bad encoding which cause misleading distances.

- Unlike HDBSCAN, it does **not support precomputed distance matrices** as robustly or flexibly.

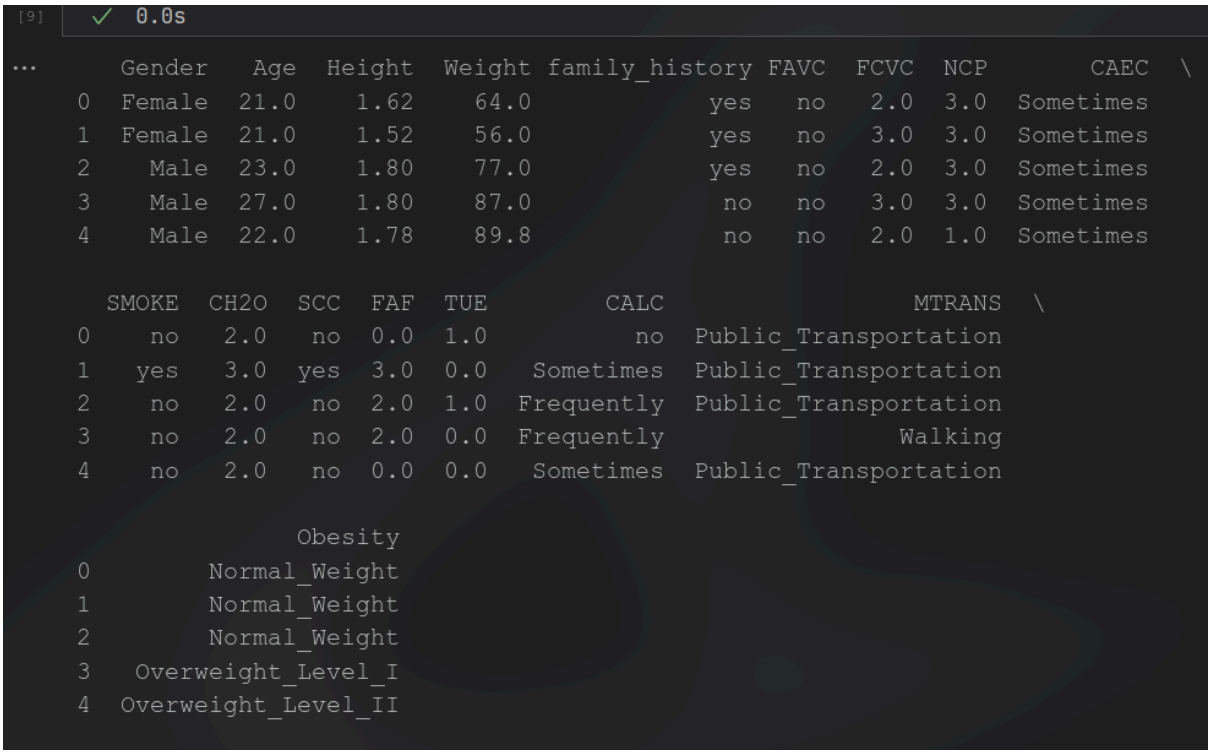
In short, when dealing with **numerical, categorical, and ordinal data**, **HDBSCAN is a more robust option than DBSCAN** especially if you can compute an appropriate distance matrix (e.g., using Gower distance or careful feature encoding). It gives better, more stable clusters in such complex datasets.

In conclusion, these models will be compared based on their ability to form **coherent, well-separated, and health-meaningful clusters**. While clustering accuracy is not evaluated the same way as in classification, internal evaluation metrics (like silhouette score) and external validation using the known "Obesity_level" will help assess the quality of clustering outcomes.

2.0 DATA PREPARATION AND PROCESSING

2.1 Loading the Dataset

```python
import pandas as pd

# Loading the dataset
df = pd.read_csv('Obesity.csv')

# Preview the data
print(df.head())
```



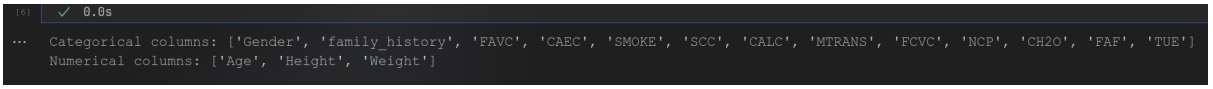
```
[9]  ✓ 0.0s

...      Gender   Age  Height  Weight family_history FAVC  FCVC  NCP       CAEC  \
    0  Female  21.0    1.62    64.0            yes   no   2.0  3.0  Sometimes
    1  Female  21.0    1.52    56.0            yes   no   3.0  3.0  Sometimes
    2    Male  23.0    1.80    77.0            yes   no   2.0  3.0  Sometimes
    3    Male  27.0    1.80    87.0             no   no   3.0  3.0  Sometimes
    4    Male  22.0    1.78    89.8             no   no   2.0  1.0  Sometimes

      SMOKE  CH2O  SCC  FAF  TUE        CALC                 MTRANS  \
    0    no   2.0   no  0.0  1.0          no  Public_Transportation
    1   yes   3.0  yes  3.0  0.0   Sometimes  Public_Transportation
    2    no   2.0   no  2.0  1.0  Frequently  Public_Transportation
    3    no   2.0   no  2.0  0.0  Frequently                Walking
    4    no   2.0   no  0.0  0.0   Sometimes  Public_Transportation

                  Obesity
    0        Normal_Weight
    1        Normal_Weight
    2        Normal_Weight
    3   Overweight_Level_I
    4  Overweight_Level_II
```

2.2 Identifying Categorical and Numerical Columns

```python
# Separate categorical and numerical columns
categorical_cols = df.select_dtypes(include='object').columns.tolist()
numerical_cols = df.select_dtypes(include=['int64',
'float64']).columns.tolist()

print("Categorical columns:", categorical_cols)
print("Numerical columns:", numerical_cols)
```

As we can see above the code has separated 'object' datatype and 'int64' or 'float64' into 2 type of columns. But if we go analyze the dataset further, we can see that 'FCVC', 'NCP', 'CH2O', 'FAF', 'TUE' are in Numerical columns, but from the dataset, they are actually *Ordinal* datatype. Hence, we need to move these columns to categorical.

```python
categorical_cols = [
    'Gender', 'family_history', 'FAVC', 'CAEC', 'SMOKE',
    'SCC', 'CALC', 'MTRANS',
    'FCVC', 'NCP', 'CH2O', 'FAF', 'TUE'  # Moving these 'Ordinal' data types
to categorical_cols
]

numerical_cols = ['Age', 'Height', 'Weight']

print("Categorical columns:", categorical_cols)
print("Numerical columns:", numerical_cols)
```



2.3 Encode Categorical Columns

2.3.1 Making sure all categorical values are preserved

```python
print(df[categorical_cols].dtypes)
```

```
[7]  ✓ 0.0s

...  Gender              object
     family_history      object
     FAVC                object
     CAEC                object
     SMOKE               object
     SCC                 object
     CALC                object
     MTRANS              object
     FCVC               float64
     NCP                float64
     CH2O               float64
     FAF                float64
     TUE                float64
     dtype: object
```

As we can see above, 'FCVC', 'NCP', 'CH2O', 'FAF', 'TUE' columns are in floats, and if we straight up put them into the encoder, the encoder will treat them as a numerical data, and weird will start to happen. As we will see in the example below.

```python
from sklearn.preprocessing import LabelEncoder

# Create a copy of the dataset
df_example_encoded = df.copy()

# Dictionary to save encoders for reverse mapping later
encoders = {}

for col in categorical_cols:
    le = LabelEncoder()
    df_example_encoded[col] = le.fit_transform(df_example_encoded[col])
    encoders[col] = le  # Saving encoder for later use

# Preview encoded data
print(df_example_encoded.head())
```

[8] ✓ 0.6s

	Gender	Age	Height	Weight	family_history	FAVC	FCVC	NCP	CAEC	SMOKE
0	0	21.0	1.62	64.0	1	0	170	477	2	0
1	0	21.0	1.52	56.0	1	0	809	477	2	1
2	1	23.0	1.80	77.0	1	0	170	477	2	0
3	1	27.0	1.80	87.0	0	0	809	477	2	0
4	1	22.0	1.78	89.8	0	0	170	0	2	0

	CH2O	SCC	FAF	TUE	CALC	MTRANS	Obesity
0	549	0	0	840	3	3	Normal_Weight
1	1267	1	1189	0	2	3	Normal_Weight
2	549	0	1071	840	1	3	Normal_Weight
3	549	0	1071	0	1	4	Overweight_Level_I
4	549	0	0	0	2	3	Overweight_Level_II

As seen in the result above, we can see 'FCVC', 'NCP', 'CH2O', 'FAF', 'TUE' values has become unpredictable. e.g. FCVC has the values 170,809 while in the original dataset below the values are 2.0,3.0 . Comparing 'FCVC', 'NCP', 'CH2O', 'FAF', 'TUE' to other categorical columns such as 'Gender', we can see that the encoder encode them correctly, e.g. Gender has the values 0 and 1, from Female and Male in the original dataset.

So to overcome this we will keep them as float64, but scale them using other encoder such as MinMaxScaler.

2.3.2 Separate Columns by Type

Python

```
categorical_cols = [
    'Gender', 'family_history', 'FAVC', 'CAEC',
    'SMOKE', 'SCC', 'CALC', 'MTRANS'
]

# Ordinal features – keep numeric, scale later
ordinal_cols = ['FCVC', 'NCP', 'CH2O', 'FAF', 'TUE']

# Other true numerical features
numerical_cols = ['Age', 'Height', 'Weight']

# Label column (for evaluation only)
label_col = 'Obesity_level'
# NOTE: No Output
```

2.3.3 Encoding Categorical Columns (The right way)

```python
from sklearn.preprocessing import LabelEncoder

df_encoded = df.copy()
encoders = {}

# Convert categorical columns to strings just in case
for col in categorical_cols:
    df_encoded[col] = df_encoded[col].astype(str)
    le = LabelEncoder()
    df_encoded[col] = le.fit_transform(df_encoded[col])
    encoders[col] = le
# NOTE: No Output
```

2.3.4 Scaling Ordinal + Numerical Columns

```python
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
scale_cols = ordinal_cols + numerical_cols  # All numeric fields to scale
df_encoded[scale_cols] = scaler.fit_transform(df_encoded[scale_cols])
# NOTE: No Output
```

2.3.5 Preparing Input Data and Categorical Indices

```python
# Combine everything except the label
X = df_encoded[categorical_cols + scale_cols].values

# Indices of categorical columns in X
categorical_indices = list(range(len(categorical_cols)))
# NOTE: No Output
```

3.0 MODEL DEVELOPMENT (UNSUPERVISED LEARNING)

3.1 Prepare the Data for K-Prototypes

```python
all_cols = categorical_cols + scale_cols

X = df_encoded[all_cols].to_numpy()

# Categorical column indices in X
categorical_indices = list(range(len(categorical_cols)))
# NOTE: No Output
```

3.1 Modelling

3.2.1 Finding Cluster Size (Elbow Method)

```python
costs = []
K = range(2, 11)

for k in K:
    kproto = KPrototypes(n_clusters=k, init='Cao', random_state=42)
    kproto.fit_predict(X, categorical=categorical_indices)
    costs.append(kproto.cost_)

# Plot
import matplotlib.pyplot as plt
plt.plot(K, costs, marker='o')
plt.xlabel('Number of clusters (k)')
plt.ylabel('Cost')
plt.title('Elbow Method for K-Prototypes')
plt.show()
```

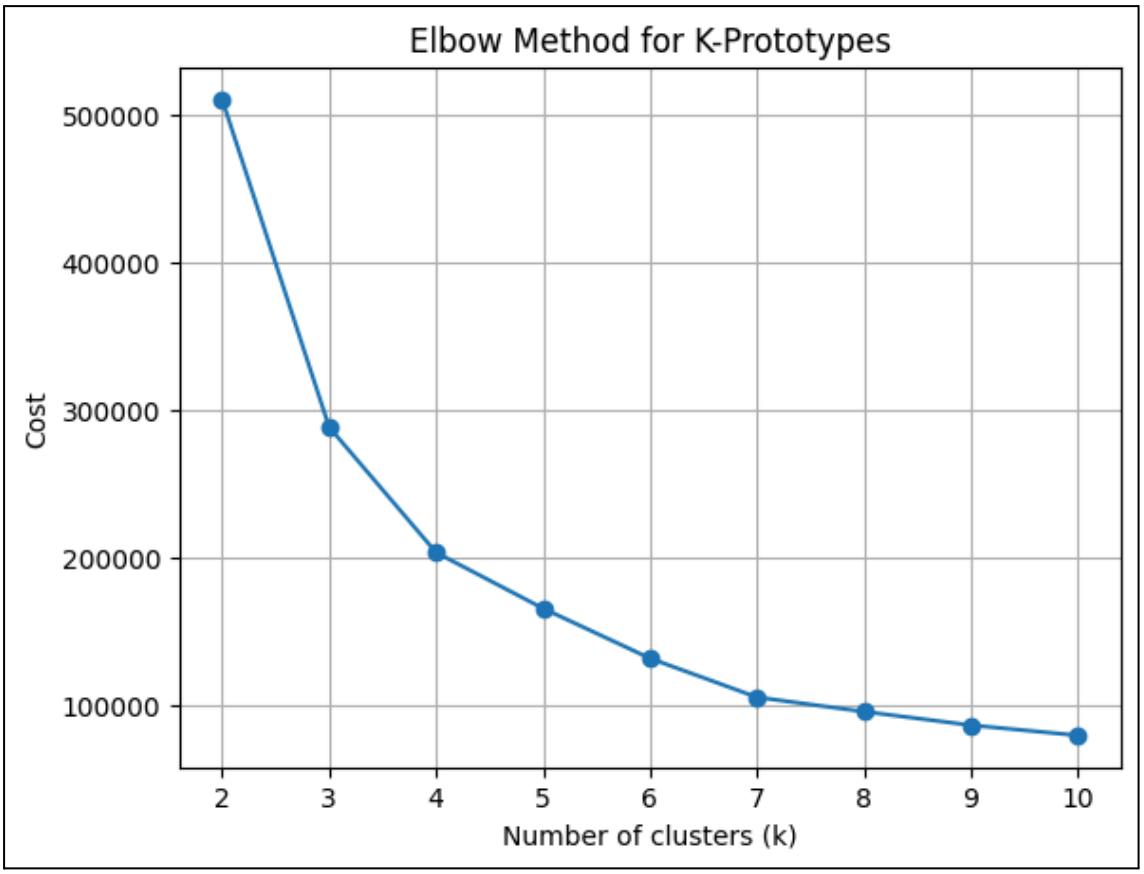


Figure 1 shows the output for the Optimal K using Elbow Methods

As shown above, the best k value is 3 before the cost of increasing k value is diminishing.

3.2.2 Clustering

```python
kproto = KPrototypes(n_clusters=3, init='Cao', verbose=2, random_state=42)
clusters = kproto.fit_predict(X, categorical=categorical_indices)
```

3.2.3 Attaching Clusters Back to DataFrame

```python
df_encoded['cluster'] = clusters

print("Cluster centroids:\n")
print(kproto.cluster_centroids_)
```

3.2.4 Visualizing Clusters (using PCA)

```python
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# Only for numeric features
pca = PCA(n_components=2)
X_pca = pca.fit_transform(df_encoded[scale_cols])

plt.figure(figsize=(8, 6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters, cmap='rainbow', alpha=0.6)
plt.title("Clusters (PCA Projection)")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.colorbar(label="Cluster")
plt.show()
```

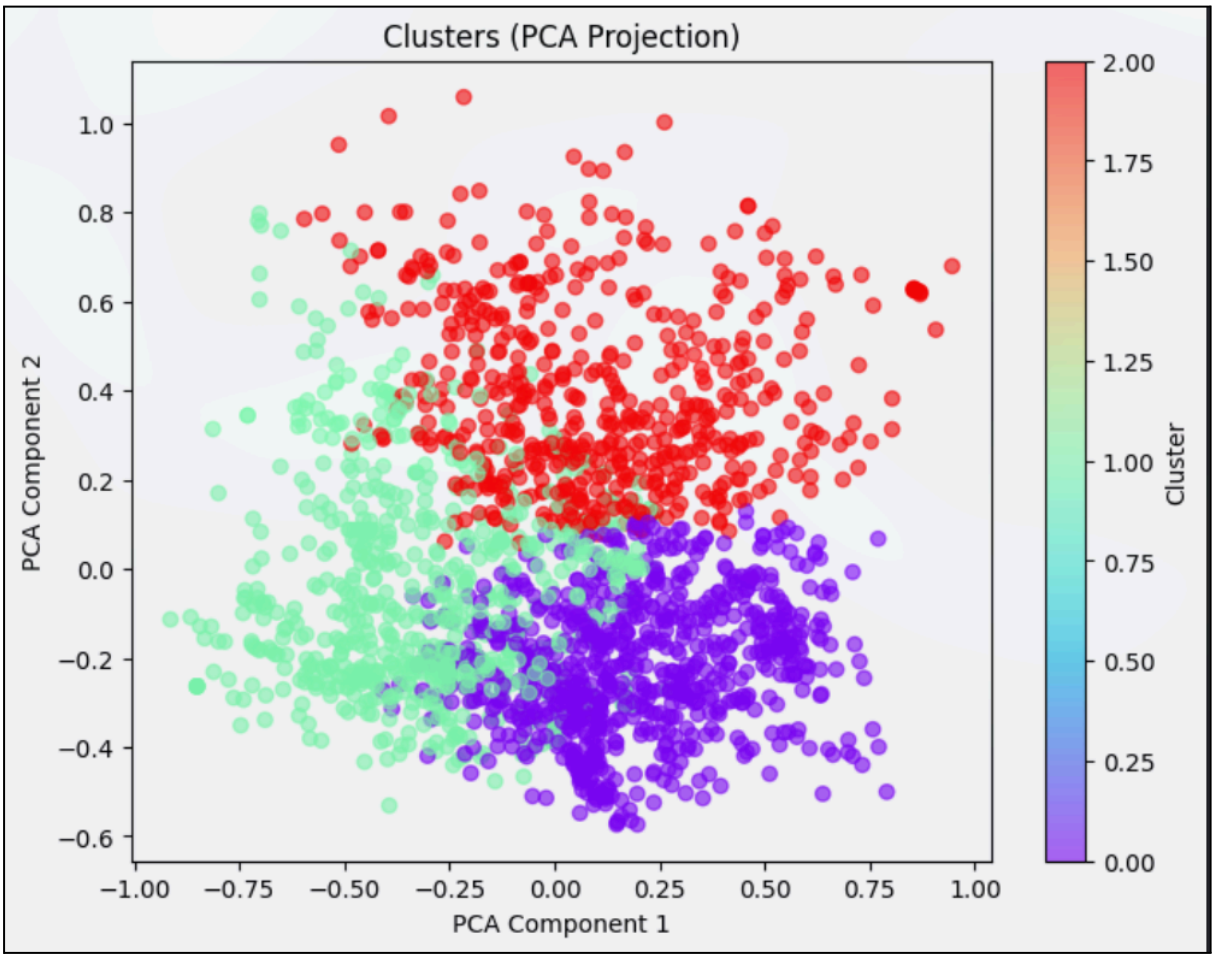


Figure 2 shows the plot of the PCA Projects for the clusters

3.3 Clusters Analysis

3.3.1 Numerical & Ordinal Columns

In the cluster analysis phase of this project. numerical and ordinal columns were analyzed together to better understand the characteristics of each cluster. Although ordinal variables represent categorical data with a natural order (such as frequency of activity or consumption), they were encoded using numerical values (e.g., 1 to 3) to reflect their ranking.

Below is the Python code for displaying the means of the Numerical and Ordinal columns.

```python
 ordinal_cols = ['Age', 'Weight', 'FAF', 'CH2O', 'FCVC', 'NCP', 'TUE']

# 1. Performing groupby and mean
cluster_means = df_encoded.groupby('cluster')[ordinal_cols].mean()

# 2. Renaming the columns in the output only (not changing original df)
cluster_means_renamed = cluster_means.rename(columns={
    'FCVC': 'Vegetable_Consumption',
    'NCP': 'Main_Meals_Per_Day',
    'CH2O': 'Water_Consumption',
    'FAF': 'Physical_Activity',
    'TUE': 'Tech_Use_Time'
})

# 3. Display it
cluster_means_renamed

# NOTE: Output to Large to put into this Report, We have Summarized them in
Table 1.
```

3.3.2 Categorical Columns

```python
for col in categorical_cols:
    print(f"Cluster breakdown for {col}:")
    print(df_encoded.groupby('cluster')[col].value_counts(normalize=True))

# NOTE: Output to Large to put into this Report, We have Summarized them in
Table 1.
```

3.3.3 Cluster Profile Summary

From the Cluster Analysis above, we have summarized the output into Table 1 below:

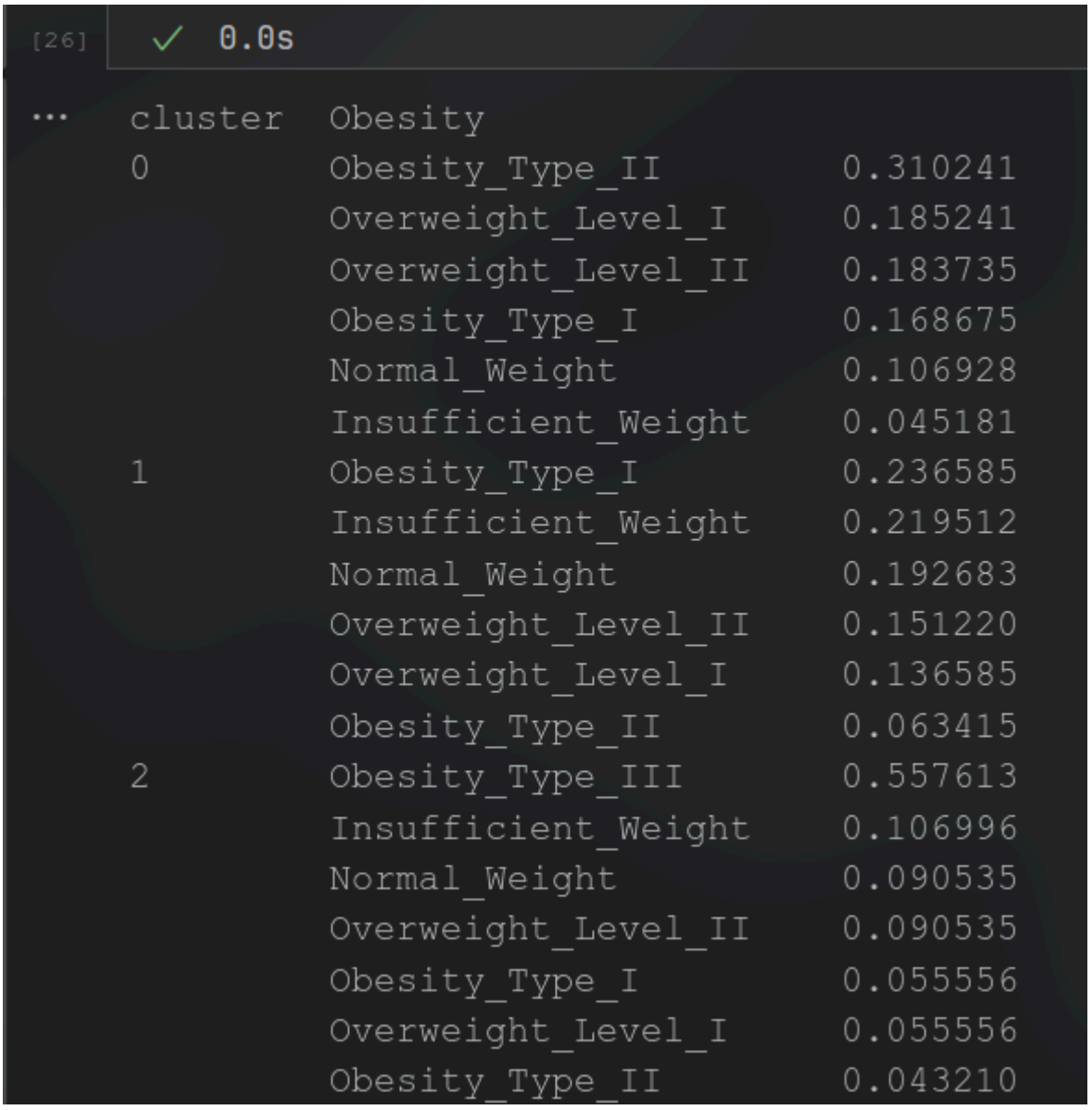
Cluster	Main Traits	Health Behavior & Lifestyle Patterns	Interpretation
0	🔹 Older individuals 🔹 Slightly heavier 🔹 Mid-to-high physical & water consumption 🔹 Mostly female (91%) 🔹 Strong family history of obesity (87%) 🔹 High fast food intake (FAVC=91%) 🔹 High snacking (CAEC=2 ~ 87%)	🔹 Moderate physical activity (0.41) 🔹 Good hydration (0.57) 🔹 Very low smoking (2%) 🔹 Frequent fast food & snacking 🔹 Low tech usage (0.12) 🔹 Majority use public transport (MTRANS=3, 65%)	**At-risk** group. Slightly healthier habits, but genetic/family risk and poor diet choices suggest an early overweight or obesity stage.
1	🔹 Youngest group (Age=0.16) 🔹 Lighter weight 🔹 More main meals (NCP=0.61) 🔹 Female majority (73%) 🔹 High tech usage (0.72)	🔹 Good water intake 🔹 Low smoking (2%) 🔹 High screen time 🔹 Lower physical activity (FAF=0.46)	Likely **students/young adults** with a sedentary lifestyle. Healthy weight now, but signs of potential weight issues ahead.
2	🔹 Higher weight (0.46) 🔹 High vegetable consumption (0.97) 🔹 Mostly male (89%) 🔹 Strong family history (88%)	🔹 Highest water intake (0.70) 🔹 Best diet score (veg + meals) 🔹 Low physical activity (0.29) 🔹 Still high snacking (CAEC=2 ~ 86%)	**Diet-conscious cluster**, likely trying to manage weight with food. Need exercise to balance out. Could include overweight individuals eating better.

Table 1 shows the Cluster Profile summary of results from the Cluster Analysis above.

3.3.4 Cluster vs Obesity(Target)

Now lets compare the clusters with the target 'Obesity' column.

```python
cluster_obesity_counts =
df_encoded.groupby('cluster')['Obesity'].value_counts(normalize=True).rename
("proportion")
print(cluster_obesity_counts)
```



3.3.5 Cluster vs Obesity Level Comparison

From the Cluster vs Obesity result above, we have summarized the output into Table 2 below:

Cluster	Most Common Obesity Levels	Top 3 Proportions	Cluster Summary
0	Obesity_Type_II (31%)	Obesity_Type_II (31%) Overweight_Level_I (18.5%) Overweight_Level_II (18.4%)	Likely represents **severely overweight** individuals with possible early obesity onset
1	Obesity_Type_I (23.6%) Insufficient_Weight (21.9%)	Mix of Obesity_Type_I, Underweight, Normal Weight	This cluster is a **mixed-risk group**, includes both **underweight** and **moderately obese** individuals
2	Obesity_Type_III (55.8%)	Obesity_Type_III (55.8%) Insufficient_Weight (10.7%) Normal Weight (9%)	Strongly skewed toward **morbid obesity**, this is your **high-risk cluster**

Table 2 shows the Cluster vs Obesity summary of results from the Cluster Analysis above.

3.5 HDBSCAN

3.5.1 Applying HDBSCAN

```python
import hdbscan

# Create and fit model
clusterer = hdbscan.HDBSCAN(min_cluster_size=30, prediction_data=True)
cluster_labels = clusterer.fit_predict(X_combined)
```

3.5.2 Visualising clusters using UMAP

```python
import umap.umap_ as umap
import matplotlib.pyplot as plt
import seaborn as sns

reducer = umap.UMAP(random_state=42)
embedding = reducer.fit_transform(X_combined)

# Visualize
df_vis = pd.DataFrame(embedding, columns=['UMAP1', 'UMAP2'])
df_vis['Cluster'] = cluster_labels

plt.figure(figsize=(10, 6))
sns.scatterplot(
    x='UMAP1', y='UMAP2',
    hue='Cluster',
    palette='tab10',
    data=df_vis,
    s=50
)
plt.title('HDBSCAN Clusters (UMAP Reduced)')
plt.legend(title='Cluster', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(True)
plt.show()
```

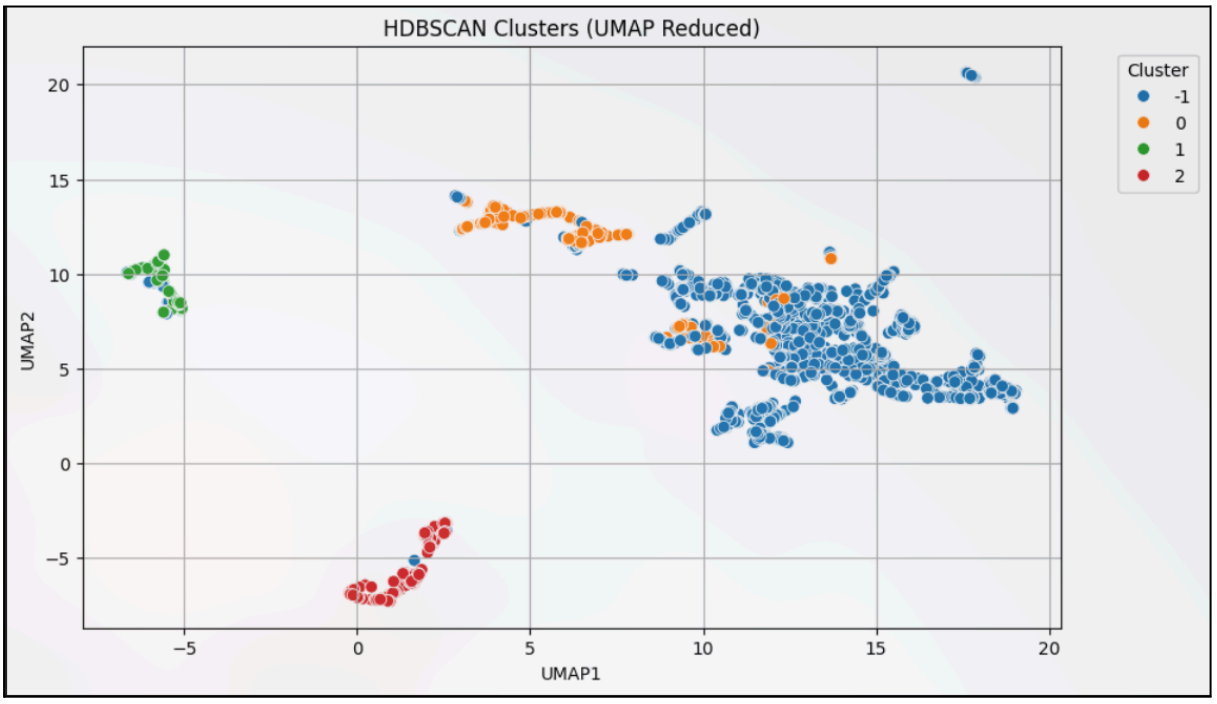


Figure 3 shows the plot of the HDBSCAN clusters using UMAP Reduced.

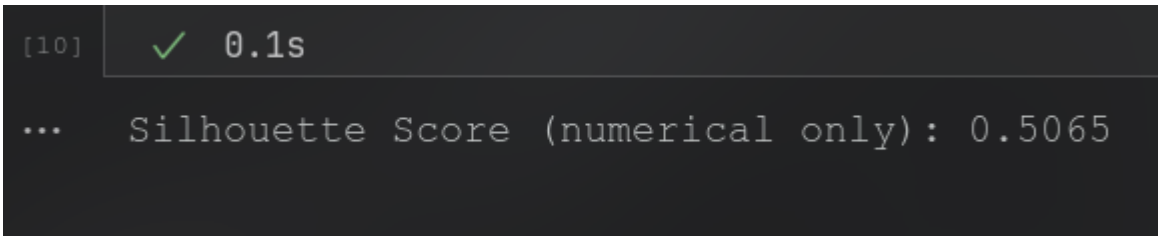
4.0 MODEL EVALUATION

4.1 Cluster Evaluation

4.1.1 K-Prototypes

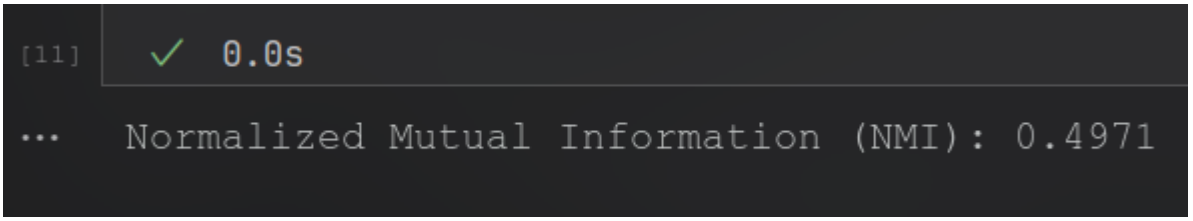
4.1.1.1 Silhouette Score (Numerical Only)

```python
from sklearn.metrics import silhouette_score

# Silhouette score only works with numerical data
# So we apply it only on numerical columns
sil_score = silhouette_score(df_encoded[numerical_cols],
df_encoded['Cluster'])
print(f'Silhouette Score (numerical only): {sil_score:.4f}')
```



4.1.1.2 Normalized Mutual Information (NMI)

```python
from sklearn.metrics import normalized_mutual_info_score

nmi_score = normalized_mutual_info_score(target, df_encoded['Cluster'])
print(f'Normalized Mutual Information (NMI): {nmi_score:.4f}')
```



4.1.2 HDBSCAN

4.1.2.1 Silhouette Score

```python
from sklearn.metrics import silhouette_score

mask = cluster_labels != -1
if mask.sum() > 0:
    sil_score = silhouette_score(X_combined[mask], cluster_labels[mask])
    print(f'Silhouette Score (excluding noise): {sil_score:.4f}')
else:
    print("No clusters found (only noise). Try lowering min_cluster_size.")
```

```
[24]
...  Silhouette Score (excluding noise): 0.3782
```

4.1.2.2 Normalized Mutual Information (NMI)

```python
from sklearn.metrics import normalized_mutual_info_score

nmi_score = normalized_mutual_info_score(target, cluster_labels)
print(f'Normalized Mutual Information (NMI): {nmi_score:.4f}')
```

```
[25]
...  Normalized Mutual Information (NMI): 0.3368
```

5.0 DISCUSSION

5.1 Cluster Analysis (K-Prototypes)

The clustering analysis in this project successfully grouped the dataset into three distinct clusters, each showing different patterns in demographics, health behaviors, and lifestyle. These findings, summarized in **Table 1**, help us understand how different groups of individuals may be at varying levels of risk for obesity based on their habits and backgrounds.

Cluster 0 mostly consists of older, slightly heavier individuals, with a large majority being female. This group shows some positive lifestyle behaviors, such as moderate physical activity and good hydration, but they also have poor dietary habits. Most of them frequently eat fast food and snack between meals. On top of that, a significant portion has a family history of obesity. Despite their efforts to stay active, the combination of genetics and poor diet makes this group vulnerable to weight gain. This is reflected in **Table 2**, where the most common obesity level in this cluster is Obesity Type II, suggesting that many of them are already experiencing moderate to severe weight-related health risks.

Cluster 1 is the youngest group and shows a more mixed pattern. The majority are female, and they tend to have healthier weights and drink enough water. However, their physical activity is relatively low, while their technology usage is high. This suggests a more sedentary lifestyle, possibly due to studying or working long hours in front of screens. Interestingly, this group also has a balanced mix of obesity categories, including underweight, normal weight, and Obesity Type I. Based on **Table 2**, this group represents a mixed-risk profile. While many are still within a healthy range, the current trends in screen time and low activity could become problematic in the future if no lifestyle changes are made.

Cluster 2 appears to be the most at-risk group. It is mostly made up of males and has the highest average weight. Members of this cluster seem to be trying to manage their health through food, as they score the highest in vegetable consumption and water intake. However, their physical activity level is the lowest among all clusters, and snacking remains high. This suggests that while they are aware of the need to eat better, they may not be incorporating enough movement into their daily routines. **Table 2** shows that over half of this cluster falls under Obesity Type III, which indicates severe obesity. This supports the idea that they are already experiencing serious weight problems, and are making efforts to improve their diet.

Overall, the clustering results offer a clearer picture of the diverse health behaviors present in the dataset. By comparing the clusters using both behavioral traits and actual obesity outcomes, we can identify which groups need more urgent interventions and which ones might benefit from preventive efforts. The use of unsupervised learning here helped uncover patterns that are not immediately obvious and provides a good starting point for future health-related research or targeted awareness campaigns.

5.2 Cluster Analysis (HDBSCAN)

The UMAP plot in **Figure 3** visualizes the clusters produced by HDBSCAN after dimensionality reduction. UMAP is used here to project high-dimensional mixed data into a 2D space, allowing us to observe how well-separated and dense each cluster is.

From the plot, we can see that **three main clusters** (Cluster 0, 1, and 2) have been identified, with each group occupying a fairly distinct region in the UMAP space. This is a good sign, as it suggests that the algorithm managed to find meaningful groupings in the data. For instance, Cluster 2 appears in the lower-left quadrant, tightly packed and isolated from the others, indicating strong intra-cluster similarity. Cluster 0 and Cluster 1 are more spread out but still visually distinct, although there is some overlap along the center-right area of the plot.

The presence of **Cluster -1**, shown in gray, represents data points that HDBSCAN labeled as **noise or outliers**. These points do not fit well into any of the main clusters, and their inclusion here highlights one of HDBSCAN's key strengths which is its ability to identify and exclude data that doesn't belong to any meaningful group. However, if the number of noise points is too high, it may suggest that the clustering parameters could be fine-tuned further.

Overall, this visualization shows that while HDBSCAN was able to form some clear and dense clusters, especially for Cluster 2, there is still some fragmentation and potential overlap between other clusters. This matches the earlier quantitative evaluation where HDBSCAN showed a moderate silhouette score (0.3782) and lower NMI (0.3362) compared to K-Prototypes, meaning that the cluster cohesion and agreement with known obesity labels were weaker.

This visualization supports the conclusion that HDBSCAN is capable of identifying structure in the data but may require further preprocessing or parameter adjustment to improve cluster clarity. Techniques like more refined feature encoding, balancing the dataset, or applying additional dimensionality reduction before UMAP could help sharpen the results in future iterations.

5.2 Model Evaluation

In this project, two clustering algorithms were used to explore and analyze patterns within the dataset: **K-Prototypes** and **HDBSCAN**. Both models were selected due to their ability to handle mixed-type data, which includes numerical, categorical, and ordinal features. The performance of these models was evaluated using two standard clustering metrics: the Silhouette Score and Normalized Mutual Information (NMI).

5.2.1 K-Prototypes

K-Prototypes is a partitional clustering algorithm designed specifically to handle datasets that contain both categorical and numerical features. It extends the K-Means algorithm by incorporating a dissimilarity function that accounts for mixed data types. In this study, K-Prototypes produced a Silhouette Score of **0.5065**, which suggests that the clusters are moderately well-separated and that the data points generally fit well within their assigned groups. The NMI score for K-Prototypes was **0.4971**, indicating a moderate agreement between the clusters generated by the algorithm and the true obesity labels in the dataset. This result is encouraging, especially given that clustering is an unsupervised task where the model does not use label information during training.

5.2.2 HDBSCAN

HDBSCAN, on the other hand, is a density-based clustering algorithm that is capable of identifying clusters of varying shapes and densities, and it can also detect noise or outliers in the data. It is especially useful for datasets where the number of clusters is not known beforehand. In this project, HDBSCAN achieved a Silhouette Score of **0.3782**, which is lower compared to K-Prototypes. This indicates that the separation between clusters was weaker, and there may have been more overlap between the data points. The NMI score for HDBSCAN was **0.3362**, suggesting that the clusters had a weaker correspondence with the known obesity levels. While HDBSCAN has strengths in flexibility and noise detection, its performance in this case was not as strong as K-Prototypes in terms of clustering consistency and alignment with the actual labels.

5.2.3 K-Prototypes vs HDBSCAN

When comparing the two models, it is clear that K-Prototypes outperformed HDBSCAN in both clustering cohesion and label agreement. The better silhouette score shows that K-Prototypes was more successful in forming compact and distinct clusters. Additionally, the higher NMI score suggests that K-Prototypes uncovered patterns in the data that were more closely aligned with the obesity categories, even without using them during training. HDBSCAN, while still providing useful groupings, showed weaker results and may have struggled due to the structure or scale of the dataset.

To improve clustering performance in future iterations, several strategies can be considered, particularly during the data preprocessing stage. One potential improvement is to apply more advanced feature scaling techniques, such as quantile transformation or robust scaling, especially for features with skewed distributions. Additionally, the quality of categorical encoding can significantly influence the model. Exploring different encoding methods such as target encoding or using specialized embeddings for categorical variables may help preserve more meaningful relationships. Another area of improvement is feature selection. Removing highly correlated or redundant features and selecting only the most informative ones can help reduce noise and improve the clarity of the clusters. Lastly, dimensionality reduction techniques like PCA or UMAP, when carefully applied to the numerical features, may help enhance separation between clusters by emphasizing the most important structure in the data.

Overall, while both models provided valuable insights, K-Prototypes offered better results for this dataset. With further tuning and more thoughtful preprocessing, future models could achieve even clearer and more accurate clustering outcomes.

6.0 CONCLUSION

This project explored the application of unsupervised learning techniques to identify patterns and group individuals based on health behavior and lifestyle features related to obesity. Using a combination of numerical, categorical, and ordinal data, clustering was performed on the Obesity Prediction dataset to uncover distinct profiles without relying on labeled outcomes during training. Through careful preprocessing and feature integration, two clustering algorithms were applied: K-Prototypes and HDBSCAN, each providing unique insights into the structure of the data.

The cluster profiles, as summarized in **Table 1**, revealed three meaningful groups. Cluster 0 represented older individuals, mostly female, who exhibited moderate physical activity and good hydration, but also showed poor dietary habits such as frequent fast food intake and high snacking. Despite these efforts, the presence of strong family history of obesity placed them at risk, which was reflected by a high proportion of Obesity Type II in **Table 2**. Cluster 1, the youngest group, showed lighter body weights and higher technology use, but lower physical activity levels. Their obesity levels were mixed, suggesting this group could be a transition point, where early unhealthy behaviors might lead to future health concerns. Cluster 2 consisted mostly of males with the highest average weight. This group had the best dietary habits, such as high vegetable and water intake, but the lowest physical activity levels. Despite their dietary improvements, a significant majority fell into the Obesity Type III category, showing that exercise remains a missing piece in managing their weight effectively.

To evaluate the performance of the clustering models, two key metrics were used, K-Prototypes achieved the best performance, with a silhouette score of **0.5065** and an NMI of **0.4971**, indicating a strong ability to group similar individuals and uncover meaningful clusters aligned with known obesity levels. In contrast, HDBSCAN achieved lower scores, with a silhouette score of **0.3782** and an NMI of **0.3362**. These results suggest that while HDBSCAN was able to detect some valid clusters and outliers, it did not perform as well in capturing the overall structure of the dataset. This was further supported by the UMAP visualization of HDBSCAN clusters, where some clusters were well separated but others showed overlap or fragmentation, and several points were classified as noise.

Overall, the project successfully demonstrated how unsupervised learning can be used to extract valuable health patterns from raw data. The findings showed that clustering algorithms, particularly K-Prototypes, can identify meaningful subgroups that reflect real-world obesity risk levels. This analysis highlights the importance of integrating multiple types of features, especially ordinal data, to capture behavioral nuances. Additionally, the evaluation suggested that future clustering efforts could benefit from further preprocessing improvements such as advanced encoding methods, better feature selection, or dimensionality reduction prior to clustering. Ultimately, this study shows that data mining techniques can play a useful role in early health risk identification and the design of targeted interventions for obesity prevention.